Reinforcement Learning for Congestion Control

A Systematic Survey of Designs, Evaluation Practice, and Barriers to Deployment

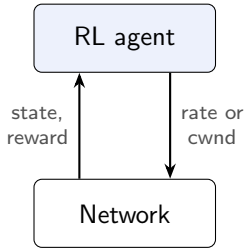
Gaurab Baral Nityashree Vangala

CS 6043: Networking

Fall 2026

Why learn congestion control?

- Classic CC (Reno, CUBIC, BBR, Copa, DCTCP) is **hand-tuned** for one kind of network.
- Wireless, satellite and data-center links break its assumptions: non-congestive loss, noisy delay, microsecond RTTs.
- RL: an agent watches throughput, delay and loss, adjusts the rate or window each interval, and is **rewarded** for high throughput and low delay.
- Aurora (2019) made deep RL the standard formulation, and the field has grown quickly since.



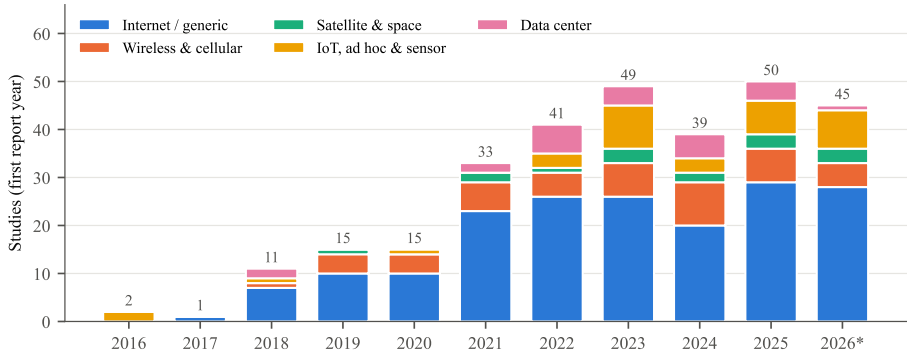
Research questions and method

- **RQ1 Design**: what do RL controllers observe, change, and learn from?
- **RQ2 Testing**: how are they trained and tested, and do they generalize?
- **RQ3 Deployment**: what stops operators from using them?

PRISMA 2020 systematic review, searches run 31 Oct 2026:

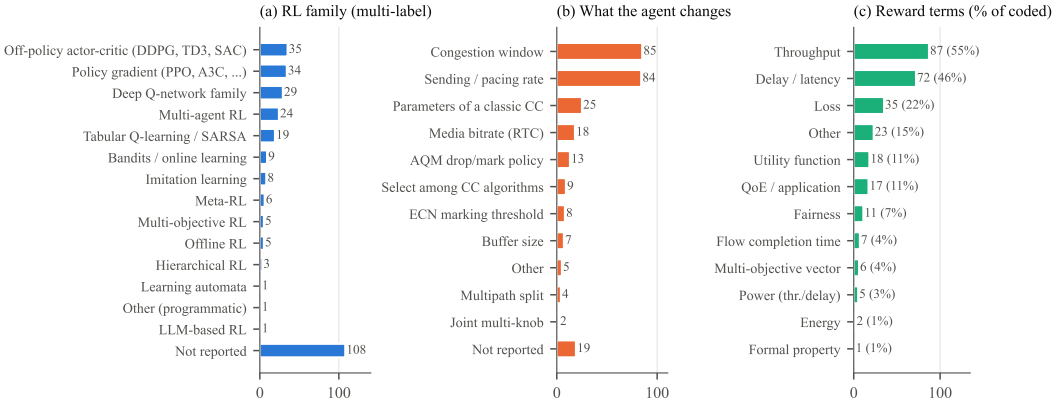
- OpenAlex (stand-in for IEEE/ACM/Scopus), arXiv, USENIX NSDI/ATC, and forward and backward citation searching
- 997 records $\rightarrow$ **301 studies** (331 reports), 2016–2026
- Coded per study: setting, control point, state, action, reward, algorithm, training environment, **highest evaluation level**, out-of-distribution testing, fairness, code
- Everything is scripted and released: every decision, every number in the paper

The field is growing fast



61% of studies appeared in 2023 or later. Settings: Internet/generic 180, wireless 48, IoT 34, data center 24, satellite 15.

RQ1: What RL controllers observe, change and optimize



Window (85) or rate (84) actions; throughput/delay/loss rewards. Only 11 rewards include fairness.

RQ1: From clean-slate policies to hybrids

Clean slate (2018–)

- Aurora (PPO), Indigo (imitation), MVFST-RL (async. in QUIC)
- Many DQN/PPO/SAC variants evaluated in ns-3

Hybrid: learning tunes, selects or bounds classic CC

- Orca: TD3 scales CUBIC's cwnd; Canopy adds certificates
- Spine, Libra, Mutant, Floo, TCP-RL

Learning from data

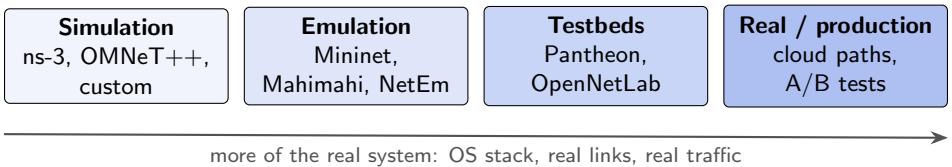
- Sage: offline RL from heuristics' logs
- RTC: offline-RL challenge on Teams logs

Trend

Hybrid share: **24%** of designs before 2023 $\rightarrow$ **35%** since.

Data centers: 57% hybrid (ECN tuning per switch, e.g., ACC).

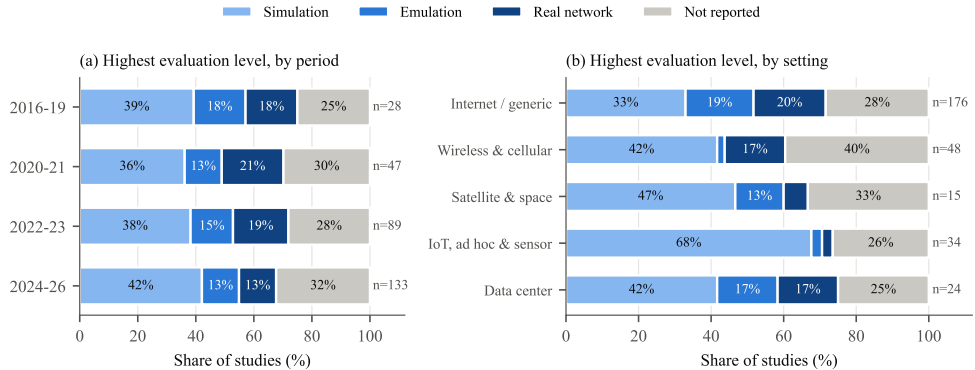
RQ2: The evaluation ladder



Correction to our proposal

Wireshark is not a simulator. It is a packet analyzer that captures and decodes real or emulated traffic. It is useful on every rung, but it does not model a network.

RQ2: Where the evidence comes from



Of studies with a known level, **57%** are simulation-only and **24%** reach a real network.
The real-network share *fell*: 21% (2020–21) → 13% (2024–26).

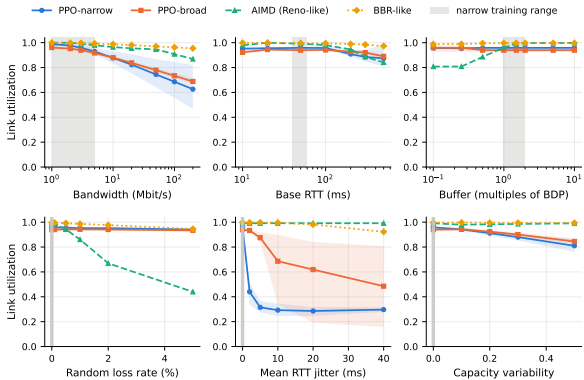
RQ2: Practices that decide whether results transfer

	All design studies		Full-text subset	
Practice	*n*	% of 266	*n*	% of 64
Code or artifact released	22	8	17	27
Tested on conditions outside the training range	30	11	13	20
Fairness or TCP-friendliness evaluated	54	20	24	38
Evaluated in emulation or on a real network	75	28	27	42
Evaluated on a real network	43	16	13	20
Evaluation environment not reported	86	32	4	6

Independent re-evaluations: public RL controllers are **unfair in almost all setups** and lag under bandwidth or latency changes; a simulator-trained PPO reached **0.5%** of a 10 Gbit/s path.

Case study: does the training distribution matter?

- Aurora-style env: delayed feedback, latency features, multiplicative rate actions
- PPO, 4 seeds $\times$ 2 training distributions:
  narrow (1–5 Mbit/s, no noise) vs.
  broad (1–100 Mbit/s, loss, jitter, drift)
- Baselines: Reno-like AIMD, BBR-like
- One factor shifted at a time



Case study: results

	In distribution		Utilization (%) under one shifted factor			
Controller	Util. (%)	Queue (RTT)	200 Mbit/s	5% loss	10 ms jitter	Worst case
PPO-narrow	96	0.03	63	94	29	29 (20 ms jitter)
PPO-broad	94	0.09	69	93	69	49 (40 ms jitter)
AIMD (Reno-like)	99	0.72	87	44	99	44 (5% loss)
BBR-like	100	0.09	95	95	100	92 (40 ms jitter)

- In distribution, the RL policy scores best on its own objective: 96% utilization with the smallest queue.
- 10 ms of RTT jitter $\Rightarrow$ **29%**: latency features read noise as congestion.
- Broad training helps only within its coverage, queues more, and 1 of 4 seeds collapsed.
- Learned and classic controllers fail *differently*, so tests must target the policy's weaknesses.

RQ3: What deployed systems have in common

SmartIW/TCP-RL (search engine, >1 year), OnRL (543 h, 151 users), Loki (101M sessions), ACC (production DCs), Floo, DuGu, Concerto, NVIDIA RL-CC (live NICs), Marten

1. **The learned part is bounded**: 6 of 9 keep a classic algorithm or safeguard in the loop; the rest imitate logs or a solver, or are distilled into trees.
2. **The operator owns the endpoints and the data**: it can A/B test, roll back, and train on its own logs.
3. **The objective is narrow**: stalls, short-flow completion time, IOPS.

No learned controller has been deployed for *general* Internet traffic.

RQ3: Nine barriers

1. Generalization under shift *partly addressed*
2. Safety, worst case *runtime fallbacks*
3. Fairness to CUBIC/BBR *open*
4. Overhead, datapath *solved in principle*
5. Interpretability *prototypes*
6. Adversarial robustness *early*
7. Objective specification *active*
8. Evaluation, reproducibility *low adoption*
9. Training cost, lifecycle *open*

Only 8% of design studies release code, and only 11% test outside their training distribution.

New direction: LLM-designed and LLM-driven controllers

- **Designer** (offline): LLMs generate controller code or utility functions, scored in an emulator (PolicySmith, GenCC; He et al.: up to 27% over BBR in a QUIC code base)
- **Translator**: natural-language intent → eBPF controller (NECC)
- **Policy** (online, triggered): CCA Reimagined, NetRL (+24% utility vs. Orca), Trailblazer (Douyin A/B test)

Assessment

The hybrid lesson is being relearned: LLMs stay off the fast path. Generated *code* is reviewable, which is good for trust. RL returns as the fine-tuning method. The evidence is still mostly emulation.

Takeaways

- The bottleneck is not the learning algorithm but the **gap between training/test conditions and real networks**.
- Hybrids and learning from logs are how RL reached production.
- Agenda: distribution-shift benchmarks, harm-based coexistence tests, validated wireless and satellite emulators, runtime assurance, offline RL from logs, datapath co-design, LLMs at design time, lifecycle operations.
- Checklist: state the evaluation level, test out of distribution (including delay noise), compare with CUBIC/BBR, report seeds and failures, release code.

Replication package: search scripts, all decisions, coded data, case study.